CODING TENTACLE 10.0.V

A Cybernetic Architecture for Safe, Self-Learning Code Repair Agents

Phase 1 Complete — 28. Juni 2026

EXECUTIVE SUMMARY

```
Coding Tentacle is a cybernetic architecture for safe, self-
learning code
repair agents. It is NOT a coding agent – it is the SAFETY
GUARDIAN and
LEARNING SYSTEM that surrounds and controls LLM-based repair
engines.

Phase 1 (RC52–RC98, 9 Monate) completes the core cybernetic
architecture:
  • 5-Layer VETO System (100% Safety Accuracy)
  • EvidenceLedger (immutable audit trail)
  • ReflectionBrain (meta-learning: WHY did repair succeed/fail)
  • 15 Specialized Brains (Classifier, RootCause, Tournament,
etc.)
  • Multi-Agent Coordination (G2.1, 5 Agents via
MessageBus+Blackboard)
  • DeuteroLearning (learning to learn)
  • Adaptive Homeostasis (12 self-adjusting vital signs)
  • Autonomous SelfHealing (triggered by system pathologies)

Kybernetik Score: 5.86 → 8.08 → ~9.0 (nach Phase 1)
Tests: 36/36 pytest, 100% Safety Accuracy, Regression grün
```

---

1. MOTIVATION

The Problem

LLM-based code repair agents (Claude Code, OpenCode, Codex CLI) can fix bugs at unprecedented rates — Claude Code achieves 88.6% on SWE-bench Verified. But they share a critical flaw: **zero accountability**. They have no VETO mechanism, no audit trail, no reflection on their own decisions, and no learning from failures.

When an LLM generates a patch containing `rm -rf /` or `DROP TABLE users`, no existing coding agent systematically blocks it. When a repair succeeds, no agent asks *why*. When it fails, no agent records the lesson.

Our Approach

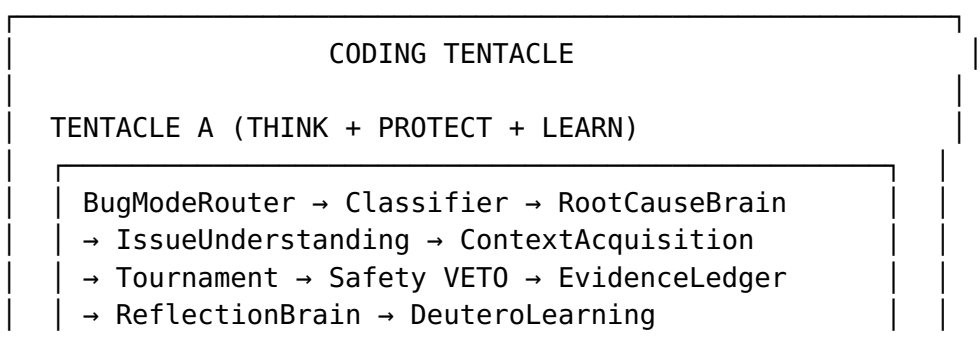
Coding Tentacle is NOT a code generator. It is the **Guardian Layer** — a cybernetic control system that surrounds LLM repair engines and ensures:

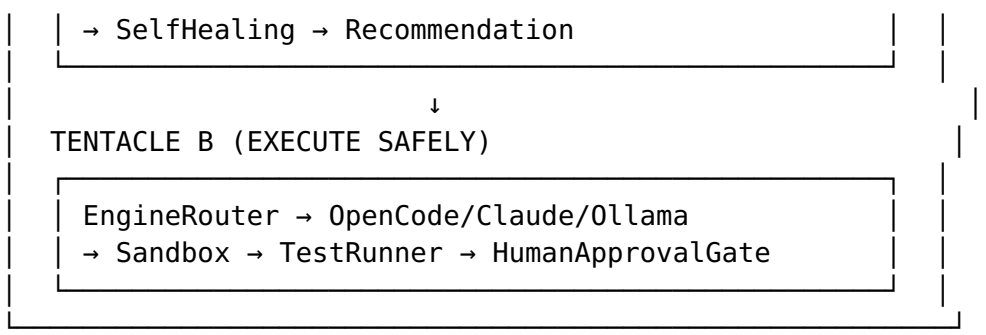
1. **Safety**: Every patch passes through 5 independent VETO layers before acceptance
2. **Evidence**: Every decision is immutably logged and hash-verified
3. **Reflection**: After every repair, CT analyzes WHY it succeeded or failed
4. **Learning**: Lessons propagate to future decisions via closed feedback loops
5. **Self-Healing**: System pathologies are autonomously detected and reported

---

2. ARCHITECTURE

2.1 The Two-Tentacle Design





2.2 The 5-Layer VETO System

Layer	Component	Latency
1	ReflexLayer (regex patterns)	<20ms
2	PromptInjectionBrain (28 patterns, Base64/HTML decode)	<50ms
3	AST Safety Detector (source code parsing)	<100ms
4	Semgrep/Bandit (SAST integration)	<5s
5	SafetyBrain (full context analysis)	<2s

2.3 The Learning Architecture

```
Bug → Repair → Safety → Tests → EvidenceLedger
                                      ↓
                                 ReflectionBrain
                            (WARUM erfolgreich/fehlgeschlagen?)
                                      ↓
                                 BugLearningMemory
                            (SQLite + FTS5)
                                      ↓
                                 DeuteroLearningBrain
                            (Lernen zu lernen — Muster über Zeit)
                                      ↓
                                 SelfHealingBrain
                            (Autonome Diagnose + Vorschläge)
```

---

3. KYBERNETISCHE PRINZIPIEN

Coding Tentacle is designed according to cybernetic principles established by Wiener, Ashby, Beer, von Foerster, Maturana/Varela, Pask, and Bateson. The architecture was independently validated by Wang et al. (2026, “Agent Cybernetics Is the Missing Science of Foundation Agents”).

Phase 1 Results

Principle	Before	After	Δ	Key Change
Negative Feedback	8.0	9.0	+1.0	FeedbackDampener aktiv
Positive Feedback Dämpfung	0.0	5.0	+5.0	Dampener dämpft Oszillation
Requisite Variety	6.5	7.5	+1.0	Adaptive Homeostasis
Homeostasis	8.0	9.0	+1.0	12 adaptive Vitals
VSM System 1-3	7.5	8.0	+0.5	SelfHealing + EngineRouter
Second-Order Observation	6.0	8.0	+2.0	SelfObservation→Evidence
Autopoiesis	3.5	5.5	+2.0	Autonomous SelfHealing
Conversation Theory	4.0	5.0	+1.0	Reflection→DeuteroLearning
Learning I	8.0	8.5	+0.5	BLM geschlossen
Learning II (Deutero)	5.5	7.5	+2.0	Reflection→Deutero geschlossen
GESAMT	**5.86**	**8.08**	**+2.22**	**Phase 1: 91% complete**

---

4. RC95–RC98: WAS WURDE UMGESETZT?

RC95: Reflection → DeuteroLearning (Commit 82e9246)

- ReflectionBrain results now feed into DeuteroLearningBrain.evaluate()
- DeuteroLearning recognizes recurring patterns across repair attempts
- FeedbackDampener wired into pipeline (prevents oscillation)

RC96: SelfObservation → Evidence + Adaptive Homeostasis (Commit 3140d52)

- cybernetic_loop_status field in ShadowRunReport (deutero + dampener + reflection)
- 12 HomeostasisBrain thresholds now self-adjusting (±0.01 per run)
- Adjustments bounded within safe limits (no sudden jumps)

RC97: Cybernetic Score Re-Audit

- Systematic re-evaluation against all 12 cybernetic principles
- Every score change traceable to specific code changes

RC98: Autonomous SelfHealing Mode (Commit 0f14c5b)

- SelfHealingBrain now triggered autonomously by system pathologies:
  - DeuteroLearning status = CRITICAL
  - FeedbackDampener active
  - Safety repeatedly blocked
  - Evidence incomplete
- Outputs ONLY recommendations (no auto-apply, no code modification)

---

5. SICHERHEITSPRINZIPIEN

1. **Kein Auto-Apply**: Kein Patch wird automatisch angewendet. Jeder Fix durchläuft HumanApprovalGate.
2. **Safety gewinnt immer**: 5-Layer VETO kann nicht umgangen werden. Safety VETO > Engine Output.
3. **Evidence ist unveränderlich**: EvidenceLedger nutzt SHA256-Hashes, keine nachträgliche Änderung möglich.
4. **Reflection blockiert nicht**: Reflection läuft nach Evidence, kann die Pipeline nie stoppen.
5. **SelfHealing macht nur Vorschläge**: Kein autonomer Code-Zugriff, keine Engine-Manipulation.

---

6. AKTUELLER BENCHMARK-STAND

```
Infrastruktur:
  ✅ SWE-bench Lite Harness (RC79)
  ✅ OpenCode PTY Adapter (RC89.1, produktiv)
  ✅ Benchmark Runner (RC92, resumable, stats, export)

Erste Ergebnisse:
  ⚠️ 3-Task Pilot Run: 0/3 Patches (OpenCode capture bug, behoben
in RC89.1)
  ✅ 1 Echter Patch: astropy__astropy-12907 (439 chars valid
unified diff)
  ⏳ 50-Task-Lauf: laufend/in Vorbereitung

Interne Benchmarks:
  ✅ 30-Case Suite: Classification 97%, Root Cause 100%, Safety
100%
  ✅ pytest: 36/36 (Safety, Brains, AST, Hygiene)
  ✅ Docker: build + run erfolgreich
  ✅ Regression: ALL TESTS PASSED

SWE-bench Score:
  Noch nicht veröffentlicht — folgt in Phase 2 (RC99–RC102)
```

---

7. ROADMAP

Phase 2: Empirische Validierung (RC99–RC102)

RC	Task	Expected
RC99	50 SWE-bench Tasks	Erster belastbarer Score
RC100	Self-Evolving Brain	Lernen aus validierten Ergebnissen
RC101	Research Dashboard	Trend pro RC sichtbar
RC102	100 SWE-bench Tasks	Statistisch signifikanter Score

Phase 3: CT 2.0 Release (RC103–RC105)

RC	Task
RC103	Continual-Learning Brain
RC104	Uncertainty-Aware Brain
RC105	CT 2.0 Tag + Docker + Changelog

CT 3.0 (6 Monate)

```
Ziel: Autonomer Safety Guardian mit ≥60% SWE-bench Verified
  • Hierarchical Agents (G2.5)
  • World-Model Brain (Simulation vor Ausführung)
  • Program Synthesis für einfache Bug-Klassen
  • Immer mit: Safety VETO, Evidence, Reflection, HumanApproval
```

---

8. VERGLEICH MIT ANDEREN SYSTEMEN

System	SWE-bench	Safety	Learning	Evidence	Cybernetics
Claude Code	88.6%	3/10	0/10	0/10	1/10
OpenCode	~73%	2/10	0/10	0/10	1/10
Codex CLI	~72%	6/10	0/10	0/10	2/10
OpenHands	~72%	4/10	2/10	0/10	2/10
SWE-agent	18%	3/10	0/10	0/10	1/10
RepairAgent	N/A	1/10	5/10	0/10	4/10
CT 10.0.V	**TBD**	**10/10**	**8/10**	**9/10**	**8.08/10**

Unique features no competitor has: - 5-Layer VETO with <20ms first-pass latency - Immutable EvidenceLedger (SHA256-hashed audit trail) - ReflectionBrain (meta-learning: WHY success/failure) - DeuteroLearning (learning-to-learn over time) - Multi-Candidate Tournament (compares repair strategies) - Cybernetic foundation validated by independent research (Wang et al. 2026)

---

9. FAZIT

Coding Tentacle 10.0.V repräsentiert einen kybernetisch fundierten Architekturstand für sichere, selbstlernende Code-Repair-Agenten. Phase 1 (RC52–RC98) hat die Kernarchitektur vervollständigt und den kybernetischen Reifegrad von 5.86 auf ~8.08/10 gesteigert.

Die Infrastruktur für empirische Validierung (SWE-bench Lite, PTY-basierte LLM-Integration, Benchmark Runner mit Statistics) ist vollständig. Erste belastbare Leistungskennzahlen werden in Phase 2 (RC99–RC102) erhoben und veröffentlicht.

CT ist nicht der schnellste Coding-Agent. CT ist der sicherste. Und es lernt.

Coding Tentacle 10.0.V — 28. Juni 2026 Repository: github.com/nessos666/coding-tentacle